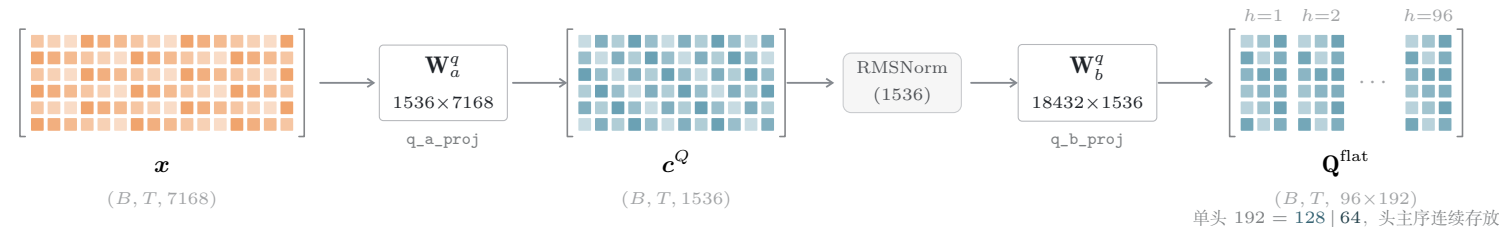


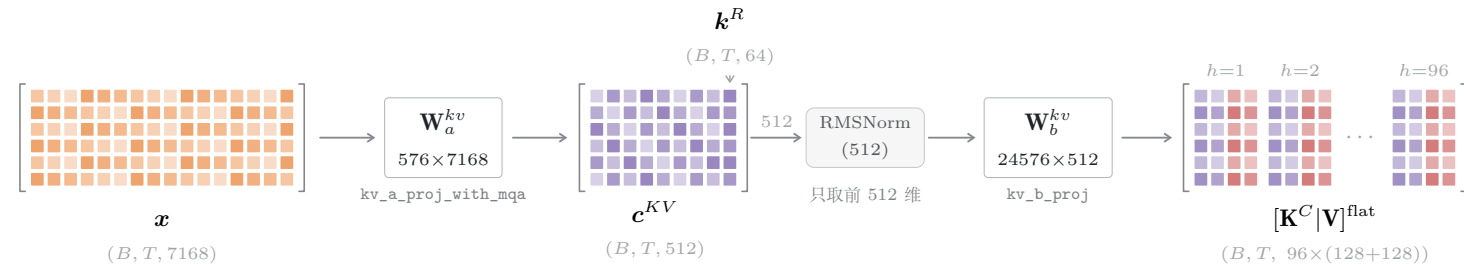
$$\mathbf{c}_t^{KV} = \mathbf{W}_a^{kv} \mathbf{x}_t, \quad [\mathbf{K}_t^C | \mathbf{V}_t] = \mathbf{W}_b^{kv} \text{RMSNorm}(\mathbf{c}_t^{KV}), \quad \mathbf{Q}_t = \mathbf{W}_b^q \text{RMSNorm}(\mathbf{W}_a^q \mathbf{x}_t), \quad \tilde{\mathbf{O}} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{192}} + \mathbf{M}\right) \mathbf{V}$$

$\mathbf{y}_t = \mathbf{W}_o [\text{Sigmoid}(\mathbf{W}_g \mathbf{x}_t) \odot \tilde{\mathbf{O}}_t]$ ——绿色即K3 新加的逐通道输出门 ($\mathbf{W}_g$ 满秩, 输入是层输入 $\mathbf{x}_t$); 全部 MLA 层用 NoPE, $\mathbf{M} \in \{0, -\infty\}$ 只做因果掩码

(a) **Query 通路**——「压缩」只是省参数的低秩分解, $\mathbf{c}^Q$ 不进任何缓存; 18432×7168 的稠密阵 (132M) 被拆成 39M



(b) **KV 通路**——一次下投影同时切出潜向量与解耦键; 只有左边 512 维被解压成 $\mathbf{K}^C$ 和 $\mathbf{V}$

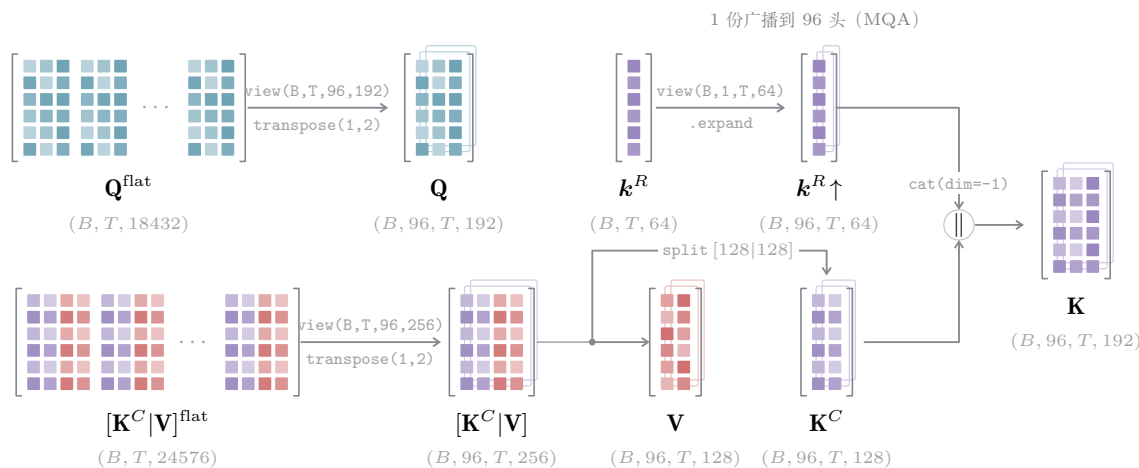


KV 缓存里到底存什么

论文口径: 每 token 每层只存 $\mathbf{c}^{KV} + \mathbf{k}^R = 512 + 64 = 576$;

KimiDynamicCache 这份参考实现图省事, 存的是展开后的 K, V: $96 \times 192 + 96 \times 128 = 30720$, 约 53×。

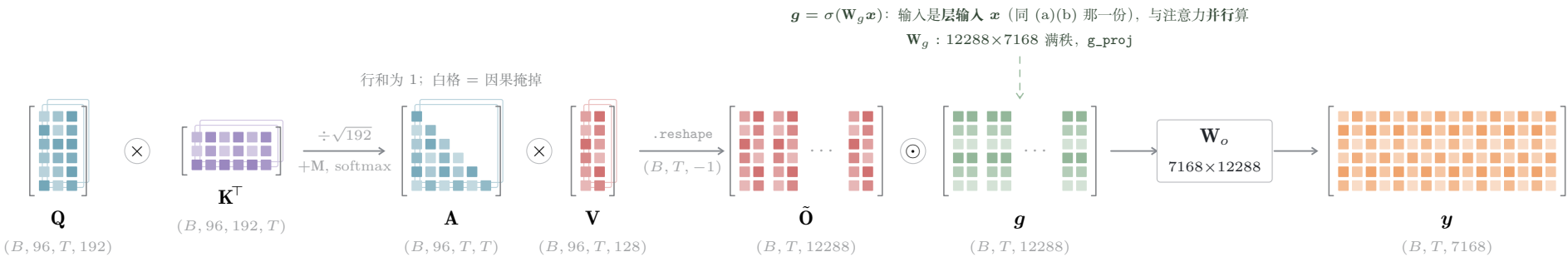
(c) **整形**——`.view(B,T,96,d).transpose(1,2)` 只改 stride 不改数据: 把扁平条按单头宽度切成 96 张面; 结果是 4 维而非 3 维



三处容易记反的地方

1. 输入 $\mathbf{x}$ 的 7168 不是 $96 \times \text{head_dim}$; 头主序的扁平形只出现在上投影之后 (18432 / 24576 / 12288)。
2. $\mathbf{k}^R$ 不是从 $\mathbf{c}^{KV}$ 解压出来的, 它是同一次下投影的另一段, 且头间共享。
3. `view+transpose` 发生在上投影之后、`split` 之前; NoPE 下 Q 的 `split`→`cat` 是恒等操作 (`rotary_emb = None`)。

(d) **NoPE 因果注意力** → 展平 → **逐通道门** → $\mathbf{W}_o$ ——门作用在展平后的 ($B, T, 12288$) 上、 $\mathbf{W}_o$ 之前



轴 B 批 (全图省略不画); T 序列位置 (面高); $h=96$ 头 (叠片深度); $d=7168$ 模型宽度 $\neq h \cdot d_v=12288$; $r_q=1536$ 、 $d_c=512$ 两个低秩瓶颈; 单头 $d_h^C=128$ (内容键) + $d_h^R=64$ (解耦键) = 192, $d_v=128$ 。

对象 $\mathbf{x}, \mathbf{y}$: 值向量。 $\mathbf{c}^Q$: 纯参数瓶颈, 不缓存。 $\mathbf{c}^{KV}$: 缓存潜向量。 $\mathbf{k}^R$: 解耦键, 只有 1 个头, `.expand` 广播到 96 (不复制显存)。 $\mathbf{A}$: 概率, $\in [0, 1]$ 、逐行和为 1、支撑为下三角。 $\mathbf{g} \in (0, 1)^{12288}$: 逐通道门, 数值不是概率。

机制 (i) `view+transpose` 是纯 stride 重排: $18432 \rightarrow (96, 192)$ 头主序, 故扁平条的分段边界就是头边界; 一次 `reshape` 再把 ($B, 96, T, 128$) 摊回 ($B, T, 12288$)。 (ii) $\mathbf{K}$ 由两段拼成: $\mathbf{K}^C$ 由 $\mathbf{c}^{KV}$ 解压 (逐头独立), $\mathbf{k}^R$ 绕过 $\mathbf{W}_b^{kv}$ 直接拼上 (全头共享); 因此 $\mathbf{Q}, \mathbf{K}$ 是 192 而 $\mathbf{V}$ 是 128。 (iii) 格数只表示轴的结构与拼合关系, 不是真实维度; 512: 128:64 = 8:2:1 按真实比例, d 与 r_q 已压缩 (否则 d 要画 112 格)。权重形状一律按 (输出×输入) 写, 与公式里的 $\mathbf{W}\mathbf{x}$ 一致, 也就等于 checkpoint 里的 `.weight.shape`。